LakeCat

covers lakecat (0.1.0)

An Engine-Close Catalog Foundation for QueryGraph

Alexy Khrabrov

chiefscientist.org

&

Codex with ChatGPT 5.5

Contents

1 LakeCat

1.1 Preface

LakeCat is a Rust-native Iceberg catalog foundation for QueryGraph. It starts from a deliberately conservative claim: the ordinary Iceberg REST catalog boundary must keep working for ordinary engines, but the next catalog also needs to become a governed control plane for Rust-first planning, semantic graph handoff, lineage, and agent access.

In this repository the catalog is LakeCat and the engine-side lakehouse implementation is Sail. The idea is not to invent a new table format or to make every client learn a new protocol. It is

to keep Iceberg compatibility at the boundary while moving the parts that need deep Iceberg knowledge closer to the Rust engine that can reason about them.

This book starts from first principles. It explains what a catalog is, why Apache Iceberg puts so much responsibility into metadata, what Sail already does with Iceberg metadata, and why a passive catalog is not enough for governed agentic systems. Then it shows the LakeCat shape: a thin Rust catalog that owns identity, tenancy, metadata-pointer state, policy gates, idempotent commits, and integration events, while delegating reusable engine, graph, and security work to Sail, Grust, and TypeSec.

The intended end state is QueryGraph.ai. LakeCat should become the catalog foundation for the next QueryGraph: a place where standard engines still see an Iceberg REST catalog, while QueryGraph can bootstrap Croissant, CDIF, OSI, ODRL, OpenLineage, TypeSec security receipts, and a Grust-backed graph from the same governed source of truth.

1.2 What A Catalog Is

A data catalog is often described as a place that lists datasets. That is true, but too small. A real catalog is the control plane between names, storage, metadata, identity, and intent.

At minimum, a catalog answers four questions:

1. What table does this name mean?
2. Where is its current metadata?
3. Who is allowed to read, write, plan, or administer it?
4. What changed, when, and under whose authority?

In a traditional database, the catalog is embedded inside the database system. The engine owns the table definitions, statistics, indexes, permissions, and transaction log. A client asks the database a question and the same system resolves names, checks permissions, plans the query, and executes it.

A lakehouse splits that system apart. Data files live in object storage. Metadata files live beside them. Multiple engines may read and write the same tables. A catalog becomes the agreement point: it maps a logical table name to the current metadata pointer and arbitrates updates to that pointer.

That pointer is deceptively important. If the catalog points at metadata version 17, the table is version 17. If a writer prepares version 18 and wins the compare-and-swap update, the table becomes version 18. If it loses, the table does not partially change. The catalog is not the table format, but it is the place where table history becomes visible and durable.

For human-scale analytics this can sound like bookkeeping. For agentic systems it becomes a trust boundary. A catalog can know the principal, the warehouse, the namespace, the table, the current snapshot, the requested columns, the row restriction, the storage profile, and the policy receipt. If that information is captured before planning and committed after state changes, the catalog becomes the control plane for governed data access rather than a passive address book.

1.3 What Iceberg Does

Apache Iceberg is a table format for large analytic tables. Its core design is simple: put the table's truth in explicit metadata files, and let engines use that metadata to plan reads and validate writes without relying on directory listing or fragile storage conventions.

An Iceberg table has a current metadata file. That metadata names schemas, partition specs, sort orders, snapshots, properties, and the current snapshot. Snapshots point to manifest lists. Manifest lists point to manifests. Manifests describe data files and delete files. The data files normally use formats such as Parquet, Avro, or ORC.

This layered metadata gives Iceberg its practical power:

- Schema evolution is explicit.
- Partition evolution is explicit.
- Snapshot isolation is explicit.
- Commit conflicts can be checked before the current pointer advances.
- Scan planning can prune manifests and files before touching data.
- Deletes can be represented without rewriting every data file immediately.
- Multiple engines can interoperate because the table state is stored in a shared, documented format.

The catalog role in Iceberg is intentionally narrow. Standard clients need to load table metadata, create namespaces and tables, commit changes, and sometimes receive credentials or scan tasks. The catalog must not require business semantics or proprietary metadata for normal table access. If standard clients have to call a non-standard endpoint to read an ordinary table, compatibility is already broken.

LakeCat preserves that rule. Iceberg metadata stays pristine. Business semantics, policy, graph, lineage, and agent state are derived control-plane or graph data. The table remains an Iceberg table.

1.4 The Current Catalog State

The current lakehouse catalog market is converging on a simple fact: the catalog is no longer a sidecar. It is the place where table identity, tenancy, commits, credentials, governance, and interoperability are negotiated.

Hive Metastore gave early lakehouses a familiar namespace and table registry, but it was born for a different format era. Cloud warehouses built proprietary catalogs around their own engines. Nessie explored Git-like table references and branching. Unity Catalog turned governance and sharing into a central product surface. Lakekeeper has shown how a modern open Iceberg REST catalog can model warehouses, projects, credentials, soft deletion, and management APIs without polluting table metadata. Polaris has made the strongest standards-centered move: an open Iceberg REST catalog with vendor gravity, clear governance ambition, and a credible route for many engines to meet at the same catalog boundary.

That is why Polaris is a winner. It is not because it is the last word in catalog architecture. It is winning because it occupies the obvious shared surface at the right moment:

- It speaks Iceberg REST instead of asking every engine to learn a new table protocol.
- It treats the catalog as a security and governance surface, not merely a pointer map.
- It gives enterprises an open center of gravity around Iceberg while the warehouse, query-engine, and cloud-storage layers remain plural.
- It can be adopted incrementally: engines can keep reading tables, while operators gain a real control plane.

LakeCat should learn from that. The winning move is not to reject Polaris-style compatibility. The winning move is to keep that compatibility and then ask what a Rust-native, QueryGraph-facing catalog can do when it is allowed to plan near Sail, emit graph through Grust, and ask TypeSec for authorization proofs.

1.5 What Intermediation Loses

Catalogs win by sitting between engines and data. That same position can also flatten the system.

When the catalog is only an intermediary, it often sees the table name, the current metadata pointer, the caller, and perhaps a credential request. The engine sees the schema, manifests, statistics, deletes, partition evolution, scan filters, and physical plan. The governance system sees policy. The lineage system sees an event after the fact. The semantic layer sees a separate model. Each system receives a shard of the truth.

The loss is not merely elegance. It is operational:

- Policy can be checked before access but not carried into scan planning.
- Credentials can be vended without proving why a raw credential exception was allowed.
- Lineage can describe that something happened but not bind to the exact governed plan, snapshot, policy, and table metadata.
- Semantic layers can drift from the physical tables they describe.
- Agents can receive broad storage access when they should have received a governed plan and short-lived, narrow task set.
- Engines can optimize with file statistics while catalogs remain blind to the consequences of those choices.

The catalog should not become a replacement engine. But it should not be a blind intermediary either. LakeCat's thesis is that the catalog can stay thin and standard while still becoming engine-close at the moments where correctness, security, and semantics depend on planning evidence.

1.6 Why Sail Matters

Sail is the Rust lakehouse engine path. It already contains the pieces that should understand Iceberg deeply: catalog abstractions, generated Iceberg REST models, DataFusion table providers, manifest pruning, metadata-as-data paths, write and commit plumbing, and format-version checks.

That matters because scan planning and commit validation are not generic catalog chores. They require knowledge of Iceberg schemas, projections, partition specs, sort orders, snapshots, manifests, data files, delete files, statistics, and expression models. If LakeCat reimplements those

details, it becomes a second Iceberg engine. The same bug class appears twice, and the catalog begins to drift away from the planner that actually executes work.

LakeCat takes the opposite path. Put reusable Iceberg and planning behavior in Sail. Let LakeCat call Sail for the parts that require engine-grade knowledge. Keep LakeCat responsible for the catalog boundary: identity, tenancy, durable state, policy gates, idempotency, audit, outbox, and standard REST compatibility.

The current LakeCat architecture follows this line. The service exposes an Iceberg REST-compatible surface under `/catalog/v1`. The Sail-facing engine path handles scan planning, table-status conversion, metadata preparation, manifest expansion, and standard response validation. LakeCat keeps only the additional context it must own: which principal asked, which policy narrowed the read, which metadata pointer was current, which commit won, and which event should be replayed to graph and lineage sinks.

1.7 Why Move Catalog Work Closer To The Engine

A passive catalog returns metadata locations and lets the client plan. That is compatible, but it is not enough for the next generation of governed systems. Agents, notebooks, services, and model pipelines need stronger guarantees:

- A policy should be enforced before a scan is planned.
- Column restrictions should narrow the projection before file tasks are produced.
- Row predicates derived from policy should become mandatory scan filters.
- A stateless `fetchScanTasks` call should not widen a prior governed plan.
- Credentials should be short-lived, scoped, and audited.
- Commit retries should be idempotent and should not replay a different request.
- Graph and lineage side effects should reflect committed catalog state, not best-effort handler side effects.

Those guarantees sit between catalog state and engine planning. If the catalog is too far from the engine, it can check a policy and then hand the client a metadata pointer, hoping the client preserves the restriction. If the engine is too far from the catalog, it can plan efficiently but may not know the governance receipt or durable identity context.

LakeCat fuses those two moments. LakeCat authorizes the request, derives the effective restriction, and asks Sail to plan or validate against the current metadata pointer. Sail performs the Iceberg work. LakeCat returns the standard shape and records the governance evidence.

The result is still an Iceberg catalog. The difference is that governed reads and writes are planned through the same Rust implementation that QueryGraph will use downstream.

1.8 LakeCat's Thin Boundary

LakeCat should be thin, but thin does not mean trivial. It owns the durable catalog state that must be correct even when external sinks are unavailable.

The core LakeCat responsibilities are:

- Serve the Iceberg REST Catalog API for standard clients.

- Model projects, warehouses, namespaces, tables, views, and storage profiles.
- Persist metadata pointers and compare-and-swap commit history.
- Validate idempotency keys and replay only matching commit bodies.
- Resolve request identity from headers, bearer tokens, agents, and TypeDID envelopes.
- Ask TypeSec for authorization decisions and persist receipts.
- Route scan planning and commit preparation through Sail.
- Record audit and outbox events inside the catalog transaction.
- Drain committed events to Grust and OpenLineage sinks.
- Publish a QueryGraph bootstrap bundle.

The deliberately excluded responsibilities are just as important:

- LakeCat does not invent a table format.
- LakeCat does not fork Iceberg manifest pruning.
- LakeCat does not own graph traversal or graph query behavior.
- LakeCat does not own security semantics or agent trust semantics.
- LakeCat does not author QueryGraph's final business semantic model.

That boundary gives each sibling project a clear job. Sail owns reusable Iceberg and planning behavior. Grust owns graph schema, storage, and query mechanics. TypeSec owns policy, capabilities, TypeDID envelopes, secure agents, and authorization semantics. QueryGraph owns the semantic application built on top.

1.9 The Read Path

The LakeCat read path begins like a standard catalog request and ends with a governed Sail plan.

1. A client asks to load or plan a table through the Iceberg REST surface.
2. LakeCat resolves the warehouse, namespace, table, and current metadata pointer.
3. LakeCat resolves the request identity.
4. LakeCat asks TypeSec whether the principal can perform the requested action.
5. TypeSec returns a decision and any enforced restrictions.
6. LakeCat turns those restrictions into a `ReadRestriction`: allowed columns, required row predicate, purpose, policy hash, and audit context.
7. LakeCat asks Sail to plan against the current metadata pointer with the effective projection and filters.
8. Sail validates Iceberg expressions against generated REST models and table schema, expands manifests, applies conservative file-bound pruning when metrics are present, and carries delete-file references into file scan tasks.
9. LakeCat returns Iceberg-compatible plan and task responses.
10. LakeCat records audit and outbox events that can later be projected into graph and lineage.

The important detail is that the policy restriction becomes part of planning, not a note beside it. An empty client projection under a column restriction means the allowed columns. A client projection can narrow further, but cannot widen. During `fetchScanTasks`, LakeCat recomputes the current restriction and requires the token to satisfy it. A stale or legacy token cannot silently expand back to all columns.

1.10 The Commit Path

The write path follows the same principle: LakeCat owns the catalog transaction, Sail owns reusable Iceberg validation and metadata preparation.

1. A client sends an Iceberg commit request, optionally with an idempotency key.
2. LakeCat validates the request shape and the idempotency key.
3. LakeCat resolves identity and asks TypeSec for the commit capability.
4. If the idempotency key already has an exact stored response, LakeCat returns that response before Sail validation or metadata-object writes.
5. LakeCat loads the current metadata pointer from the store.
6. LakeCat delegates Iceberg update validation and metadata assembly to Sail.
7. LakeCat rejects metadata-object writes that target the table's current metadata pointer, so the current metadata file cannot be overwritten before the store commit has won.
8. LakeCat rejects metadata-write plans that do not carry a concrete new metadata location.
9. LakeCat writes the new metadata object through the warehouse storage profile.
10. LakeCat advances the table pointer with compare-and-swap.
11. LakeCat persists idempotency, audit, pointer-log, and outbox records.
12. If the store rejects the commit after a local metadata write, LakeCat cleans up the uncommitted metadata object when it can do so safely.
13. Outbox draining projects the committed event to graph and lineage sinks.

The cleanup path is deliberately secondary to the commit result. If metadata cleanup fails after the store rejects a commit, LakeCat preserves the original store or compare-and-swap error class and appends cleanup context. A stale pointer conflict should still look like a conflict to an Iceberg client.

Idempotency is part of correctness. Reusing the same key for the same commit can return the stored response even after the table has advanced beyond the original commit requirements. Reusing the same key for a different body must conflict. LakeCat persists a normalized request hash and stores only audit-safe evidence, not raw secrets or raw idempotency keys. The service regression for this path proves the replay happens before metadata-object writes: an exact retry returns the stored response without touching the already committed metadata object. Another regression sends a commit whose requested metadata location is the table's current pointer and verifies that LakeCat returns a bad request without touching the existing metadata file. The same guard fails closed if a future Sail plan asks LakeCat to write metadata but does not provide a new object location.

Commit records also carry a response hash over the stored table response. That pair matters: the request hash proves which commit body won or replayed, while the response hash proves which metadata pointer and table body LakeCat returned to clients and later projected through graph and lineage replay.

The same commit record includes compact summary evidence: Iceberg format version, current snapshot id, and the policy hash from the authorization receipt when one exists. QueryGraph can

inspect those fields from the pointer-log/outbox stream without parsing full table metadata for every catalog audit question.

Operators and QueryGraph can read that pointer-log evidence through a governed management endpoint:

```
curl -s \
  -H 'x-lakecat-principal: operator@example.com' \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/tables/
events/commits
```

The response contains compact commit records: sequence number, previous and new metadata locations, request hash, response hash, idempotency-key hash, Iceberg format version, current snapshot id, policy hash, principal, and a commit hash. The read itself enters the durable outbox as `table.commits-listed` and drains as lineage evidence, so audit tools can prove who inspected pointer history without requiring direct access to the Turso catalog database. QGLake acceptance now exercises this path directly: the fixture issues an idempotent no-op commit probe, reads the compact commit-history endpoint, verifies that the record preserves the table's Iceberg format-version and current snapshot summary, and then requires the lineage drain to replay `table.commits-listed` receipt hashes plus compact commit count, sequence-number, and commit-hash summary fields before the QueryGraph handoff is accepted.

1.11 The Durable Spine

LakeCat's durable local spine uses the Rust `turso` crate behind the `turso-local` feature. The store contract remains portable, but the local foundation is not SQLx. The important tables are not an application afterthought; they define the catalog's control-plane memory:

- projects and warehouses;
- storage profiles;
- namespaces and tables;
- metadata pointer log;
- idempotency records;
- soft deletes;
- policy bindings;
- audit events;
- outbox events.

Object storage remains the source of Iceberg metadata files. Turso stores the atomic pointer, management state, idempotency evidence, and event record. This mirrors the Iceberg catalog contract: metadata files describe the table; catalog state decides which metadata file is current.

1.12 Grust For Graph Concepts

Catalog events naturally form a graph. A warehouse contains namespaces. A namespace contains tables. A table has columns, snapshots, manifests, files, policies, commits, scan plans, principals, and lineage runs. QueryGraph needs that graph, but LakeCat should not become a graph database.

Grust owns the graph layer. It is the place for reusable graph taxonomy, projection builders, graph stores, traversal indexes, Cypher support, and typed or untyped graph operations. LakeCat's responsibility is narrower: translate committed catalog events into a bounded envelope and pass it through a catalog-facing sink.

In practice this means LakeCat can emit graph events for stable catalog facts:

```
Warehouse CONTAINS Namespace
Namespace CONTAINS Table
Table HAS_COLUMN Column
Table GOVERNED_BY Policy
Principal CAN_PLAN ScanPlan
Commit DERIVED_FROM Snapshot
LineageRun USED_BY QueryGraphModel
```

High-cardinality file and manifest facts should stay queryable through Iceberg/Sail metadata-as-data unless Grust provides a reusable taxonomy and storage strategy for them. The graph should be powerful, but the catalog must not smuggle a second lakehouse engine into its event sink.

The current local direction already proves the boundary: LakeCat's `grust-local` sink calls Grust-owned LakeCat projection helpers, and the Grust Cypher boundary test verifies catalog graph projection without making LakeCat own Cypher parsing, traversal, or graph execution.

1.13 TypeSec For Security

LakeCat is a policy enforcement point, not the author of security semantics. TypeSec owns the semantics: RBAC, ODRL-style policy composition, typed capabilities, TypeDID envelopes, secure agents, credential issuance decisions, and authorization proofs.

Every externally meaningful action should pass through TypeSec:

- catalog configuration reads;
- namespace creation and listing;
- table creation, load, scan planning, commit, drop, and restore;
- credential vending;
- policy management;
- graph and lineage reads.

LakeCat gathers the request context and asks TypeSec for a decision. The context can include principal DID, agent DID, bearer-derived subject, warehouse, namespace, table, columns, snapshot, requested credential duration, purpose, and active policy bindings. TypeSec returns a decision and receipt. LakeCat persists the receipt with audit-safe hashes and applies the resulting restrictions before Sail plans.

This is where ODRL becomes operational. An ODRL-style policy can say that a principal may read only certain columns, only for a purpose, or only with an enforced row predicate. LakeCat parses the minimal enforceable subset it needs to narrow the plan, but policy composition and authorization semantics belong to TypeSec. LakeCat should ask TypeSec, not grow a parallel security language.

Credential vending follows the same rule. Raw credential vending is an audited exception. Governed Sail-planned reads are the default path for agents and untrusted principals. When credentials must be issued, TypeSec checks the `credentials.issue` capability for the exact secret reference and LakeCat returns only scoped, short-lived credential configuration.

1.14 Rust-First Engines And The V3 To V4 Path

The new Rust-first engine path matters because it changes where catalog intelligence can live. Sail is not a Java service with Rust bindings bolted on the side. It is a Rust engine path built around Arrow, DataFusion, generated Iceberg REST models, catalog provider traits, manifest pruning, metadata-as-data scans, and table-status conversion.

That shape gives LakeCat a better option than reimplementation. LakeCat can ask Sail for typed Iceberg behavior instead of parsing just enough JSON to survive a request. That matters for Iceberg v3 and the emerging v4 work. Format v3 already pushes catalogs and engines toward richer metadata, row lineage, deletion semantics, and better interoperability around advanced table state. Format v4 is still settling, but it is plainly moving the lakehouse toward more adaptive and structured metadata rather than less.

LakeCat should evolve under three rules:

1. Conform to Iceberg v3 for ordinary clients.
2. Preserve unknown and emerging v4 metadata without claiming settled semantics.
3. Prefer typed Sail support as soon as Sail exposes it, using JSON passthrough only as a compatibility bridge.

That gives LakeCat room to support v4-ready capability flags, round-trip tests, metadata extension preservation, and future metadata-tree planning without forking Iceberg. The catalog can become more intelligent while the table remains portable. The standard path stays boring. The governed Sail path becomes richer.

The current v4 bridge is intentionally narrow and tested as such. When LakeCat sees `format-version: 4`, it does not pretend that Sail already has a settled typed v4 model. Instead, `lakecat-sail` extracts the stable JSON envelope fields that remain useful across versions: table UUID, location, schema id, snapshot id, sequence number, manifest-list path, default spec, and field names. It can plan a governed manifest-list scan task from that envelope and validate the signed plan task again during `fetchScanTasks` so a stateless fetch cannot drift to a different manifest list or widen the governed projection and filters. It also validates stable commit requirements such as table UUID, current schema id, main snapshot id, last assigned field id, and default spec id. Pruning and typed metadata-tree semantics wait for Sail-owned v4 support.

1.15 OSI, OpenLineage, And Responsible Semantic Handoff

QueryGraph needs more than physical table access. It needs a semantic picture: datasets, fields, policies, lineage, graph relationships, and anchors that can survive import. LakeCat should publish that picture without pretending to be QueryGraph.

The LakeCat bootstrap bundle contains:

- Semantic Croissant and CDIF projections for dataset and field discovery.
- An OSI handoff with stable dataset and field anchors.
- ODRL policy artifacts and TypeSec policy context.
- OpenLineage events for catalog changes, scan plans, commits, and maintenance.
- A Grust-ready graph envelope.
- A manifest that hashes each emitted artifact.

The OSI boundary is deliberately careful. LakeCat should not author rich business metrics, dimensions, joins, ontology claims, or authoritative semantic names. It can publish stable anchors and governed source metadata. QueryGraph owns the final semantic model.

This is the Responsible Semantic Layer boundary. Semantic Croissant and CDIF make datasets and fields discoverable and exchangeable. OSI gives QueryGraph a stable handoff for semantic anchors without forcing LakeCat to own business meaning. OpenLineage records how catalog and planning events happened. Together they let the semantic layer be responsible because it can be traced back to catalog state, policy, and lineage, not just to a hand-authored model file.

OpenLineage fits the catalog outbox. A committed table create, scan plan, commit, soft delete, restore, or maintenance action can become a lineage event. Because the event is drained from a durable outbox after the catalog transaction, lineage reflects committed state rather than a handler's best-effort side effect.

1.16 QueryGraph.ai When LakeCat Is Done

QueryGraph.ai is the enterprise lakehouse this work is pointing toward. QueryGraph needs the catalog to be more than a storage address book. It wants to answer questions over data, metadata, semantics, policy, and lineage as one governed graph. That requires a catalog foundation that can speak to ordinary Iceberg clients and also publish trustworthy control-plane facts.

LakeCat supports that foundation by exposing a QueryGraph bootstrap endpoint:

```
/querygraph/v1/bootstrap
```

The bundle gives QueryGraph an import contract. QueryGraph can verify hashes, load catalog graph envelopes through Grust, inspect policy artifacts through TypeSec, and attach semantic modeling work to stable dataset and field anchors. The import path should prove that LakeCat is the substrate, not a standalone demo.

When LakeCat is done, the QueryGraph.ai architecture looks like this:

```
Standard engines and tools
  Spark, Trino, Flink, PyIceberg, notebooks
    |
    | Iceberg REST
    v
LakeCat catalog
  identity, tenancy, metadata pointers, commits, policy gates,
  idempotency, credential-vending decisions, audit, durable outbox
    |
    | typed planning and table semantics
```

```
    v
Sail
  Rust-native Iceberg planning, metadata-as-data, scan pruning,
  delete handling, commit validation, table maintenance
    |
    | graph events                  | authorization proofs
    v                               v
Grust                           TypeSec
  catalog graph, traversal,        RBAC, ODRL, capabilities,
  projection, graph stores         TypeDID, secure agents
    |
    | semantic and lineage bootstrap
    v
QueryGraph.ai
  Responsible Semantic Layer over Croissant, CDIF, OSI,
  OpenLineage, ODRL, graph, and governed table access
```

Basic catalog use remains optional and standard. A normal Iceberg engine can load and commit tables without knowing QueryGraph exists. The enhanced path is there for enterprises that want more: governed Sail-planned reads, TypeSec authorization receipts, ODRL rights, TypeDID agent identity, OpenLineage replay, Semantic Croissant/CDIF publication, OSI handoff, and Grust graph loading.

That is the core motivation for LakeCat. The next QueryGraph.ai should not bolt governance, graph, and lineage onto tables after the fact. It should begin from a catalog that already records governed state transitions and exposes standard, engine-close planning.

1.17 Implementation Shape

The current workspace shape expresses the architecture directly:

```
crates/
  lakecat-core        stable IDs, errors, time, config, content hashes
  lakecat-api         Iceberg REST request/response adapters
  lakecat-store       catalog state traits and Turso-backed implementation
  lakecat-sail        Sail provider bridge and privileged planning client
  lakecat-graph       catalog-facing Grust sink/adapters
  lakecat-security    TypeSec integration and authorization receipts
  lakecat-lineage     OpenLineage projection and event receipts
  lakecat-querygraph  Croissant/CDIF/OSI/ODRL/OpenLineage bootstrap projection
  lakecat-service     axum service, middleware, auth, routing
  lakecat-cli         admin, local demo, conformance, bootstrap export
```

Feature gates keep integrations honest:

```
sail-local    use local Sail APIs for planning and provider integration
typesec-local use local TypeSec APIs for governance and TypeDID verification
grust-local   use local Grust APIs for catalog graph projection
turso-local   use the Turso-backed durable store
```

Embedded defaults stay safe for tests. Real integrations are explicit. That matters because LakeCat is a foundation, not a pile of optional demos. A test that only uses memory stores should not accidentally depend on a sibling repo. A test that claims to validate TypeSec should enable `typesec-local` and call TypeSec.

The local dependency contract is executable because LakeCat still depends on active sibling work. Grust and TypeSec are versioned local path dependencies: LakeCat resolves them from `../grust` and `../typesec` while also pinning the published crate versions it expects. Sail is different today: LakeCat uses local Sail paths plus a checked-in patch bridge for helper APIs that are not yet published. Before pushing a slice that touches integration features, run:

```
scripts/check-local-dependency-contract.sh
```

The script checks the manual-only CI trigger, the Grust/TypeSec versioned path pins, the local Sail path bridge, and the Sail patch files manual CI applies. It is not a substitute for upstreaming the Sail helper APIs or re-enabling automatic CI; it is a guard that makes drift visible while LakeCat still lives across these sibling repositories.

1.18 Standard Compatibility And Extensions

LakeCat must be boring where standards require boring behavior. Standard Iceberg clients should be able to use the catalog without knowing QueryGraph exists. Business semantics and agent state must not be required custom Iceberg metadata.

Extensions belong beside the standard path:

- `/catalog/v1` serves Iceberg REST compatibility.
- management APIs handle warehouses, policies, profiles, and operational state.
- `/querygraph/v1/bootstrap` publishes QueryGraph import artifacts.
- feature-gated Sail paths provide governed scan planning and local provider integration.

Format v4 work should follow the same rule. Prefer typed Sail support when available. JSON passthrough can bridge compatibility, but it is not the desired long-term implementation. Round-trip tests should prove LakeCat preserves unknown or evolving metadata without claiming settled semantics too early.

1.19 Workflow Examples

The catalog is easiest to understand by watching it participate in ordinary work. LakeCat should not ask users to think about graph, lineage, security, and Sail every time they read a table. Those systems should appear when they matter: at the boundary where a name is resolved, a policy is enforced, a plan is created, credentials are withheld or issued, and a durable event is replayed.

The examples below use one table, `local.default.events`, but the pattern is the same for larger warehouses. The important point is not the exact sample data. It is the catalog role in each workflow.

1.19.1 Starting The Catalog

A local operator starts LakeCat as an Iceberg REST catalog plus management surface:

```
cargo run -p lakecat-service --features sail-local,turso-local,typesec-
local,grust-local
```

The standard catalog path is still `/catalog/v1`. The management and QueryGraph surfaces sit beside it:

```
/catalog/v1
/management/v1
/querygraph/v1/bootstrap
```

A simple health-oriented configuration read shows the split. Standard engines care about the Iceberg endpoints. Operators and QueryGraph care about the management and bootstrap endpoints.

```
curl -s http://127.0.0.1:3000/catalog/v1/config
```

At this point the catalog is already doing more than route HTTP. It has a warehouse identity, a store, a governance engine, a Sail planning seam, a graph sink, and a lineage sink. Embedded defaults keep the local loop small, but the same trait boundaries can point to Turso, TypeSec, Grust, and Sail.

1.19.2 Registering The Warehouse Shape

An operator usually starts with management objects. A server groups projects. A project groups warehouses. A warehouse owns namespaces, tables, views, storage profiles, policy bindings, and the metadata pointer state that standard engines see through Iceberg REST.

```
curl -s -X PUT http://127.0.0.1:3000/management/v1/servers/prod \
  -H 'content-type: application/json' \
  -d '{
    "display-name": "Production LakeCat",
    "endpoint-url": "https://lakecat.example.com",
    "properties": {
      "owner": "platform"
    }
  }'

curl -s -X PUT http://127.0.0.1:3000/management/v1/projects/resilience \
  -H 'content-type: application/json' \
  -d '{
    "display-name": "Resilience Desk",
    "server-id": "prod",
    "properties": {
      "environment": "demo"
    }
  }'

curl -s -X PUT http://127.0.0.1:3000/management/v1/projects/resilience/
warehouses/local \
  -H 'content-type: application/json' \
  -d '{
    "display-name": "Local QGLake Warehouse",
```

```
    "storage-root": "file:///tmp/lakecat/qglake",
    "properties": {
      "querygraph": "enabled"
    }
  }'
```

These writes are not Iceberg table metadata. They are catalog control-plane state. LakeCat persists them durably, records authorization receipts, and writes outbox events. When the outbox drains, project and warehouse changes become catalog graph events; project, warehouse, server, and storage-profile changes also become OpenLineage receipts. QueryGraph can later learn the management shape without requiring every Iceberg client to understand it.

1.19.3 Storage Profiles And Credential Roots

Storage profiles bind a warehouse to physical storage roots and credential issuance policy. A local profile can return scoped local file configuration. A remote profile should usually reference a secret store and require TypeSec to authorize issuance before any resolver sees the secret reference.

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/storage-profiles/local-
events \
  -H 'content-type: application/json' \
  -d '{
    "location-prefix": "file:///tmp/lakecat/qglake/events",
    "provider": "local",
    "issuance-mode": "standard",
    "properties": {
      "purpose": "developer-loop"
    }
  }'

curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/storage-profiles/s3-events
\
  -H 'content-type: application/json' \
  -d '{
    "location-prefix": "s3://lakecat/events",
    "provider": "s3",
    "issuance-mode": "secret-ref",
    "secret-ref": "vault://kv/lakecat/events",
    "public-config": {
      "region": "us-west-2"
    },
    "properties": {
      "purpose": "production-events"
    }
  }'
```

The catalog row stores the public profile and secret reference, not raw cloud keys. A later credential request is checked against TypeSec and against the effective read restriction for the target table. Agents with fine-grained table restrictions are steered to governed Sail-planned reads instead of raw credentials. Trusted humans can receive audited standard credentials only when policy allows the exception.

1.19.4 A PySpark User Reads Iceberg

A PySpark user should not need to know about QueryGraph. They configure Spark's Iceberg REST catalog and point it at LakeCat:

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("lakecat-events")
    .config("spark.sql.catalog.lakecat", "org.apache.iceberg.spark.SparkCatalog")
    .config("spark.sql.catalog.lakecat.type", "rest")
    .config("spark.sql.catalog.lakecat.uri", "http://127.0.0.1:3000/catalog/v1")
    .config("spark.sql.defaultCatalog", "lakecat")
    .getOrCreate()
)

events = spark.table("default.events")
events.select("event_id", "severity").where("severity = 'critical'").show()
```

For an unrestricted principal, the flow looks like a normal Iceberg read:

1. Spark asks LakeCat to resolve `default.events`.
2. LakeCat checks the principal and table capability.
3. LakeCat returns an Iceberg-compatible table response.
4. Spark plans the read using Iceberg metadata.

For a governed principal, the more interesting path is server-side planning. The user request still looks ordinary, but LakeCat derives the mandatory restriction before Sail sees the plan. If policy allows only `event_id` and `severity`, then a wider client projection is narrowed:

```
client asks:     event_id, severity, raw_payload
policy allows:   event_id, severity
Sail receives:   event_id, severity
```

The catalog does not trust the client to remember that. The restriction is re-applied when scan tasks are fetched, and the audit payload records the policy hash, narrowed columns, row predicate, storage location, metadata location, and principal.

1.19.5 A Notebook Requests Credentials

Credential vending is deliberately different from scan planning. Returning storage credentials gives the client broader power than returning a governed task list, so LakeCat treats it as an exception path.

```
curl -s \
  -H 'x-lakecat-principal: agent:triage' \
  http://127.0.0.1:3000/catalog/v1/local/namespaces/default/tables/events/
credentials
```

For an agent bound by a fine-grained restriction, LakeCat should fail closed:

```
{
  "credentials": [],
  "lakecat:credential-block-reason": "fine-grained read restriction requires
Sail-planned reads"
}
```

That empty credential response is not a missing feature. It is the intended agentic posture. The agent should ask LakeCat to plan the read through Sail, not receive raw storage reach. The audit event records the decision and the lineage outbox can replay the credential-vend attempt with the same block reason.

For a trusted human principal, policy can allow an audited raw credential exception. LakeCat still records the same context:

```
principal: analyst:maya
principal-kind: human
table: local.default.events
decision: raw credential exception allowed
reason: trusted human principal
restriction: present in receipt
lineage: credential vend attempted
```

The contrast matters for operators. They can prove that agents were kept on the governed path while a human exception was explicit, policy-backed, and replayable.

1.19.6 A View Becomes Part Of The Catalog Story

Views are catalog objects too. LakeCat stores durable view records with SQL, dialect, schema version, typed columns, properties, creator, and warehouse scope. They can be managed through the management API or through warehouse-prefixed catalog routes.

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view \
  -H 'content-type: application/json' \
  -d '{
    "sql": "select event_id, severity from default.events where severity =
'\''critical'\''",
    "dialect": "spark-sql",
    "schema-version": 1,
    "columns": [
      {
        "name": "event_id",
        "data-type": {"type": "long"},
        "nullable": false,
```

```
        "comment": "Stable event identifier"
      },
      {
        "name": "severity",
        "data-type": {"type": "string"},
        "nullable": false,
        "comment": "Operational severity"
      }
    ],
    "properties": {
      "owner": "resilience-desk"
    }
  }'
```

This is not Iceberg business metadata glued onto a table. It is catalog state about a view object. LakeCat records `view.listed`, `view.upserted`, `view.loaded`, and `view.dropped` audit events. Outbox replay projects listing reads to namespace-scoped graph and OpenLineage evidence, and projects single-view changes and reads to catalog-facing View graph events plus LakeCat OpenLineage view dataset receipts. QueryGraph bootstrap can then include views with OSI hashes, store-assigned view versions, view-aware graph edges, and OpenLineage view counts. The lineage-drain summary also carries compact view replay identity:

```
{
  "name": "events_view",
  "view-version": 1,
  "schema-version": 1,
  "dialect": "sql"
}
```

That `view-version` is assigned by the durable store on each upsert, not by the caller. It is the first compatibility bridge toward Iceberg view commit history: QueryGraph can compare a bootstrap view artifact with the catalog's current view version today. LakeCat also writes a compact view-version receipt in the durable store. The receipt records the stable view id, assigned version, previous version, previous receipt hash, content hash, principal, operation, and timestamp. That makes the compact receipt list a hash chain: version 2 points at the version 1 receipt hash, and a later tombstone points at the last upsert receipt hash. Fuller version-log semantics remain a Sail-aligned implementation target. When a view is dropped, LakeCat appends a compact tombstone receipt instead of inventing a new view version: the receipt keeps `view-version` at the last durable version, sets `operation` to `drop`, links to the previous receipt, and preserves the last content hash so QueryGraph or an operator can prove which catalog view state was removed.

```
{
  "event-type": "view.upserted",
  "view-warehouse": "local",
  "view-namespace": ["default"],
  "view-name": "events_view",
  "view-stable-id": "lakecat:view:local:default:events_view",
  "view-version": 1,
```

```
  "graph-events": 2,
  "lineage-events": 1,
  "replay-event-hashes": ["sha256:..."],
  "replay-open-lineage-hashes": ["sha256:..."]
}
```

A `view.listed` replay is intentionally namespace-scoped. It records the warehouse, namespace, view count, graph and lineage projection counts, and receipt hashes for the list read without fabricating a single `view-stable-id`.

QGLake acceptance compares both that `view-stable-id` and `view-version` with the accepted QueryGraph bootstrap view artifacts. That closes a small but important gap: the bootstrap bundle may say a view was exported, and the drain evidence can now prove the corresponding view catalog event was replayed at the same durable catalog version with graph and lineage receipts.

When QueryGraph bootstrap is replayed through the outbox, LakeCat includes only compact receipt hashes:

```
{
  "event-type": "querygraph.bootstrap",
  "view-artifact-count": 1,
  "view-version-receipt-hashes": ["sha256:..."]
}
```

QGLake acceptance requires one non-empty receipt hash for each accepted view artifact. That keeps normal Iceberg view access standard, but gives the QueryGraph handoff a durable proof that the exported view version came from LakeCat's catalog spine. The fixture also exercises the deletion side of the same workflow: it creates a transient view, accepts a QueryGraph bootstrap that contains that view, drops the view, reads the receipt chain through the governed management endpoints by view name and namespace, and then requires lineage-drain replay to include `view.dropped`, `view.version-receipts-listed`, and `view.version-receipt-chains-listed` evidence with non-empty tombstone receipt hashes and namespace chain hashes. LakeCat also validates the ordered `previous-receipt-hash` links before marking a namespace chain as `chain-verified`, so QueryGraph can reject a replay that contains hashes but not a coherent chain.

QueryGraph and operators can also read the compact receipt chain directly from the governed management surface:

```
curl -s \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view/version-receipts \
  -H 'x-lakecat-agent-did: did:example:resilience-agent'
```

```
{
  "receipts": [
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "view-version": 1,
      "previous-view-version": null,
      "operation": "upsert",
```

```
      "view-hash": "sha256:...",
      "receipt-hash": "sha256:..."
    },
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "view-version": 1,
      "previous-view-version": 1,
      "previous-receipt-hash": "sha256:...",
      "operation": "drop",
      "view-hash": "sha256:...",
      "receipt-hash": "sha256:..."
    }
  ]
}
```

The response is catalog evidence, not Iceberg table metadata. It lets QueryGraph verify the version chain, including tombstones after the current view row is gone, while keeping the richer view history model available for a future Sail-owned implementation.

When the caller needs discovery rather than a known view name, the management surface can return all view receipt chains in a namespace, including chains for views that no longer appear in the active view list:

```
curl -s \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/view-
version-receipt-chains \
  -H 'x-lakecat-agent-did: did:example:resilience-agent'
```

```
{
  "chains": [
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "chain-hash": "sha256:...",
      "chain-verified": true,
      "latest-view-version": 1,
      "latest-operation": "drop",
      "tombstoned": true,
      "receipt-count": 2,
      "receipts": ["..."]
    }
  ]
}
```

That tombstone read is replayable too. LakeCat projects `view.version-receipts-listed` and `view.version-receipt-chains-listed` as lineage evidence, not as graph topology. The graph taxonomy stays in Grust; LakeCat only proves that the governed read saw the tombstone receipt needed to explain why a previously accepted view is now deleted. The namespace response also carries a deterministic `chain-hash` over the chain identity and ordered receipt hashes, and lineage-drain summaries replay that value as `view-version-receipt-chain-hashes` together with a verified-chain count. The QGLake fixture now fails if the namespace-level receipt-chain read, its

chain hash, or its verified-chain evidence is absent from lineage-drain replay, so QueryGraph acceptance can depend on compact chain evidence without scraping store internals.

1.19.7 QueryGraph Bootstrap

QueryGraph should import LakeCat facts through a verified handoff, not by scraping service internals. The bootstrap endpoint publishes a bundle with artifact hashes:

```
curl -s \
  -H 'x-lakecat-principal: agent:querygraph-importer' \
  http://127.0.0.1:3000/querygraph/v1/bootstrap \
  -o target/qglake/lakecat-bootstrap.json
```

The bundle contains catalog tables, views, policy bindings, graph artifacts, OpenLineage artifacts, Croissant/CDIF/OSI/ODRL projections, and a manifest that hashes what was emitted. The manifest is the import contract. QueryGraph can refuse a bundle whose graph hash, OpenLineage hash, table artifact hash, view artifact hash, or QueryGraph import-compatibility hash does not match. For view-bearing bundles, the import contract also carries compact receipt evidence for each exported view version:

```
{
  "querygraph-import": {
    "schema-version": "lakecat.querygraph.import-compat.v1",
    "view-count": 1,
    "view-receipt-evidence": [
      {
        "stable-id": "lakecat:view:local:default:events_view",
        "view-version": 1,
        "receipt-hash": "sha256:..."
      }
    ],
    "view-receipt-evidence-hash": "sha256:..."
  }
}
```

That gives QueryGraph a manifest-covered way to reject a view bootstrap that lost the catalog receipt chain before the richer graph import begins.

The QueryGraph side should verify the same bundle before importing it:

```
cd /Users/alexy/src/querygraph/qg-rust

cargo run -- lakecat-verify \
  --bundle /Users/alexy/src/lakecat/target/qglake/lakecat-bootstrap.json

cargo run -- lakecat-import \
  --bundle /Users/alexy/src/lakecat/target/qglake/lakecat-bootstrap.json \
  --output .querygraph/lakecat/import-plan.json
```

The importer checks the outer bundle hash, the manifest hashes, the QueryGraph-import compatibility hash, the graph hash, and view receipt evidence. The graph envelope must be valid as

a graph, not just valid JSON: for example, a table and a view in the same namespace must share one namespace node, not emit duplicate vertex ids. That validation belongs on the QueryGraph/Grust side, while LakeCat is responsible for producing a clean catalog-facing graph projection.

For the full local handoff, LakeCat carries a script that runs both sides without writing generated artifacts into the QueryGraph checkout:

`scripts/qglake-handoff-local.sh`

The script starts LakeCat on `127.0.0.1:18181`, uses a Turso-backed local store under `target/qglake-handoff/`, generates the paired QGLake bootstrap bundle and lineage-drain response, runs `qglake-verify-replay`, then runs QueryGraph's `lakecat-verify` and `lakecat-import` against the same bundle. The resulting import plan is written to `target/qglake-handoff/querygraph-import-plan.json`. The same run also writes `target/qglake-handoff/handoff-summary.json`, a compact machine-readable record under the `lakecat.qglake.handoff-summary.v1` schema. It records the catalog URL, principal, table scope, LakeCat replay status from `lakecat.qglake.replay-verification.v1`, QueryGraph-verified table/view counts, and semantic bundle/graph/OpenLineage/import hashes plus standards accepted only after LakeCat replay, `lakecat-verify`, and `lakecat-import` agree. It also records structured scan, management, credential, and table-commit replay evidence, artifact paths, raw file hashes, captured LakeCat replay output, captured QueryGraph verification output, captured QueryGraph import output, and service log path. That makes the handoff repeatable from the LakeCat repo while keeping QueryGraph responsible for graph validation and import semantics.

This gives the semantic layer a responsible starting point. LakeCat says:

```
Here are the governed catalog objects.
Here are the policies that shaped planning.
Here are stable dataset and field anchors.
Here is the graph envelope.
Here is the OpenLineage replay evidence.
Here are the hashes that bind the handoff.
```

QueryGraph then owns the richer semantic work: metrics, dimensions, joins, business names, multi-dataset reasoning, and agent-facing synthesis.

1.19.8 Draining The Outbox

LakeCat records side effects as durable outbox events. Draining the outbox is what turns committed catalog facts into graph and lineage receipts:

```
curl -s -X POST \
  -H 'x-lakecat-principal: agent:lineage-drainer' \
  http://127.0.0.1:3000/management/v1/lineage/drain \
  -o target/qglake/lineage-drain.json
```

A useful drain response includes delivered event types, graph projection counts, lineage projection counts, receipt hashes, and the authorization proof for the drain request itself. That last part is easy to overlook. Reading the replay stream is also privileged, so LakeCat records that the drainer was allowed to read lineage evidence. Standard catalog reads replay too:

`catalog.config-read` records a warehouse-scoped graph/OpenLineage fact for the Iceberg REST configuration entrypoint; `namespace.listed` records the namespace listing at the warehouse; and `namespace.loaded` records the specific namespace resolved through the standard catalog route. For view events, the response includes the warehouse, namespace, view name, and QueryGraph-compatible stable id, so downstream acceptance can check replay identity without parsing the full audit payload. Table restores replay as table graph evidence plus a restore OpenLineage receipt, so a soft-deleted table returning to service is visible to QueryGraph without forcing LakeCat to invent restore-specific graph taxonomy. Management list reads for policy bindings, projects, servers, storage profiles, and warehouses replay as OpenLineage receipts too. They intentionally do not create list-specific graph nodes in LakeCat; Grust owns the reusable hierarchy and traversal model. The drain response lifts their counts and management scope into compact fields, so QueryGraph can verify the control-plane read evidence without opening the raw lineage payload. The QGLake acceptance workflow now establishes its server/project/warehouse tenant spine, performs governed server, project, warehouse, policy-list, storage-profile-list, scan-planning, scan-task-fetch, and table commit-history reads before bootstrap, and rejects a drain that does not replay matching `server.listed`, `project.listed`, `warehouse.listed`, `policy-binding.listed`, `storage-profile.listed`, `table.scan-planned`, `table.scan-tasks-fetched`, and `table.commits-listed` evidence. For scan replay, the typed drain summary carries scan-plan task counts plus fetched file-scan, delete-file, and child-plan task counts; for commit-history replay, it carries the commit count, committed sequence numbers, and commit hashes. This lets QueryGraph verify the governed Sail-planned read and pointer-history inspection without parsing the full lineage payload.

For handoff testing, LakeCat can verify a saved bootstrap bundle and a saved drain response together:

```
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
```

That offline check replays the same boundary assertions used by the live QGLake fixture: the bundle manifest must verify, the QueryGraph import compatibility contract must match, and the lineage drain must carry matching bootstrap hashes, credential-denial receipts, management-list evidence, scan/fetch task evidence, and table commit-history receipt evidence, plus view receipt evidence when views are present. On success, the command prints the accepted bundle and QueryGraph import hashes, table/view counts, and compact control-plane lines such as:

```
scan replay plan_tasks=1 file_tasks=1 delete_files=1 child_plan_tasks=1
management replay servers=1 projects=1 warehouses=1 policies=1 storage_profiles=1
credential replay restricted=blocked:sail-planned-read-required
restricted_count=0 human=allowed:trusted-human-audited-raw human_count=1
table commit history commits=1 sequences=1 hashes=sha256:...
```

Those lines are intentionally small enough for QueryGraph handoff scripts and operator logs, but they still come from the same typed lineage-drain summaries that the verifier requires before accepting replay.

The end-to-end result is a chain:

```
catalog write
  -> audit event
  -> outbox event
  -> graph projection
  -> OpenLineage projection
  -> QueryGraph import evidence
```

If graph or lineage sinks are down, catalog state should not be lost or rolled back accidentally. The outbox lets LakeCat retry projection from committed state.

1.19.9 An Agentic QGLake Flow

The agentic path is the reason LakeCat has to be more than a passive catalog. Imagine a resilience supervisor agent investigating incidents:

1. The supervisor delegates table triage to a specialist agent.
2. The specialist asks LakeCat to plan a scan over `local.default.events`.
3. LakeCat resolves the agent identity and TypeDID context.
4. TypeSec authorizes the table scan and returns a restricted capability.
5. LakeCat narrows the projection and appends the required row predicate.
6. Sail plans against the current Iceberg metadata and delete manifests.
7. LakeCat returns governed plan and fetch-task responses.
8. The specialist summarizes only the allowed result shape.
9. LakeCat records scan and credential decisions into audit/outbox.
10. QueryGraph imports graph, policy, lineage, and bootstrap evidence.

The key point is the absence of raw storage reach. The specialist agent does not need broad cloud credentials to do its job. It needs a governed plan, a bounded task set, and a receipt trail.

The local fixture compresses this story into a short artifact-producing sequence:

```
cargo run -p lakecat-cli -- qglake-fixture \
  --output target/qglake/lakecat-bootstrap.json \
  --drain-output target/qglake/lineage-drain.json \
  --principal did:example:agent
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
```

The one-command handoff wraps the same evidence in a live local service run and then asks QueryGraph to verify and import it:

```
scripts/qglake-handoff-local.sh
cat target/qglake-handoff/handoff-summary.json
```

That fixture creates the sample table shape, installs a restricted policy, verifies governed scan planning, verifies fetch-scan-task reapplication, exercises delete manifest handling, probes credential-vend behavior for agents and trusted humans, verifies compact table commit-history evidence, exports QueryGraph bootstrap artifacts, drains the outbox, and proves the resulting bundle through QueryGraph's Rust verifier/importer. It is small, but it is not decorative. It is the acceptance story for a catalog that participates in the user workflow from notebook to agent. The summary file gives automation a single stable place to find the accepted table/view counts, semantic hashes, bundle, lineage drain, import plan, captured verifier outputs, and raw artifact hashes without scraping terminal text.

1.20 Operating The Book's Example System

The local development posture is intentionally small:

```
cargo run -p lakecat-cli -- config
cargo run -p lakecat-cli -- storage-profile-list
cargo run -p lakecat-cli -- policy-list
cargo run -p lakecat-cli -- qglake-fixture \
  --output target/qglake/lakecat-bootstrap.json \
  --drain-output target/qglake/lineage-drain.json \
  --principal did:example:agent
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
scripts/qglake-handoff-local.sh
cargo run -p lakecat-cli -- bootstrap-export \
  --output lakecat-bootstrap.json
```

The important thing is what these commands exercise. They are not a separate product surface. They touch the same catalog, policy, bootstrap, and QueryGraph export contracts that the service uses.

1.21 What Comes Next

LakeCat is a direction more than a single release. The next slices should keep making the architecture more true:

1. Remove temporary Sail patch bridges when the required helpers are available upstream.
2. Keep pushing reusable Iceberg table-status, planning, metadata-as-data, and commit helpers into Sail.
3. Make the transactional outbox the only path for graph and lineage side effects.
4. Expand TypeSec-backed capability checks until every privileged path is unbypassable.
5. Keep graph taxonomy and Cypher behavior in Grust.
6. Keep OSI as a QueryGraph-owned semantic handoff rather than a LakeCat-authored business model.
7. Prove the bootstrap bundle through QueryGraph import on every meaningful public-surface change.

The payoff is a catalog foundation that feels ordinary to Iceberg clients and rich to QueryGraph. Tables remain portable. Policies become enforceable. Graph and lineage become replayable. Sail plans close to the data. QueryGraph gets a trustworthy substrate for the next version of its lakehouse intelligence.